MENTOR-RL Methods

Peter Kruse

June 2026

0.1 Introduction and Motivation

0.1.1 Biological Interpretation as Mechanistic Discovery

Biological interpretation involves the development and discovery of mechanistic relationships between biological entities such as genes, proteins, RNA, and other molecular components. Interpretation is essential for discovering context-specific relationships and developing models of disease, phenotypes, pathways, and regulatory systems. Recently, machine learning (ML) and artificial intelligence (AI) techniques such as iterative random forest (iRF) [7, 11] and related approaches have accelerated research through the construction of biologically informed networks, which scientists explore to generate mechanistic and functional hypotheses.

As these methods increase the resolution and scope of networks, biological interpretation has become increasingly mediated by large, multi-scale graph structures. To comprehensively examine these large biological networks and supporting databases, mechanistic exploration requires iterative querying and multi-step reasoning. Recent agentic interpretation systems such as eubiota [27] and MENTOR-IA [53] demonstrate that provenance-preserving retrieval and tool-mediated synthesis can yield high-fidelity mechanistic narratives. However, these pipelines depend on frontier model inference and are not fully optimized as a trainable policy.

0.1.2 Biological Interpretation is Bottlenecked by Human Cognition and Frontier Model Cost

Although ML/AI methods such as MENTOR [50] have refined mechanistic focus into groups of functionally related modules of genes, these modules can still be massive and intractable for individual researchers to systematically explore. Additionally, most research emphasizes a deep-but-narrow focus on a small subset of functionally related genes, which can overlook broader, yet biologically meaningful, network context, particularly in highly interactive systems such as living organisms. As biological networks grow in complexity, the combinatorial space of possible multi-hop interactions expands rapidly, while human interpretive capacity remains fixed.

While large language models (LLMs) can synthesize large volumes of information, frontier inference cost scales with exploration length and reasoning depth, limiting agentic deployment for long-horizon network traversal. This mismatch creates a scaling limit in biological mechanistic interpretation: rich relational structures can be computed, but downstream synthesis remains constrained by human bandwidth and model cost. Consequently, the speed, breadth, and contextual depth of mechanistic discovery are limited, particularly for complex traits, polygenic diseases, and multi-layer regulatory systems.

0.1.3 AI-Guided Reasoning to Extend Mechanistic Interpretation Beyond Human Scale

Open-source LLMs present an opportunity to aid scientists in mechanistic discovery tasks without being cost prohibitive. Recent developments in agent-based systems enable models to invoke deterministic tools with structured interfaces, substantially reducing hallucination and enabling provenance-preserving exploration. We propose MENTOR-RL, a reasoning agent trained on biological ground truth and verifiable interaction objectives, enabling scalable mechanistic exploration without reliance on proprietary models.

By training a transformer-based agent with reinforcement learning across tool-use tasks and contextual levels of biological network data, we aim to produce a model that can traverse complex networks, synthesize *multi-hop* mechanistic hypotheses, and integrate both local functional structure and global topological context. Leveraging open-source model offerings allows local deployment, with training costs amortized across many exploration sessions. Rather than replacing scientific judgment, MENTOR-RL extends the interpretive bandwidth of researchers, enabling broader, faster, and more context-aware mechanistic discovery.

We make five contributions:

1. We formalize mechanistic graph exploration as a sequential decision process with a single policy operating in two modes: an actor that proposes reasoning and tool calls, and a verifier that updates a structured hypothesis and emits a continuation decision.

2. We construct a dual-source module training corpus, comprising MENTOR-derived dendrogram modules and RWR++ elbow-filtered random-walk-with-restart lines-of-evidence (RWR-LOE) modules, supporting four task cases: partial group completion, mechanism explanation, noisy group refinement, and recognition of no shared mechanism.

3. We design a deterministic, schema-grounded reward system over structured state transitions, enabling reproducible scoring without proprietary judges at the step level during preference optimization and the trajectory level for policy gradient optimization.

4. We propose a staged training pipeline, including warm-start SFT, DPO on shared-prefix trajectory branches, and GRPO for long-horizon exploration. We pair this with a curriculum that scales task difficulty, noise, and trajectory length.

5. We will release the training environment, evaluation harness, and trained checkpoints, and we will benchmark against frontier and open models under both empirical and blind expert-preference evaluation.

0.2 Background and Related Work

0.2.1 Biological Networks and Mechanistic Ground Truth

Systems Biology and Mechanistic Discovery

Systems biology examines biological function by studying interactions between molecular entities such as genes, proteins, and metabolites, rather than analyzing their individual properties [19, 6]. A central goal is identifying *modules*, or groups of functionally related entities, and characterizing the mechanisms that underlie their behavior. Because network interactions can span multiple scales and regulatory layers, modern systems biology relies on multiplex and multilayer network representations to capture context-specific relationships [5, 22]. Module-level analysis has become a foundational tool for identifying disease mechanisms, prioritizing drug targets, and interpreting polygenic phenotypes [29], motivating the need for curated, mechanistic ground truth data against which computational methods can be developed and evaluated.

Curated Interaction Maps

Large-scale systems biology depends on curated molecular interaction databases that capture experimentally supported relationships at varying levels of resolution. Creating high-coverage, expert-annotated networks is crucial for enabling large-scale systems biology studies and encouraging mechanistic discovery. BioGRID [35] catalogs physical and genetic interactions across humans and model organisms from both low- and high-throughput studies. Reactome [30, 16] provides manually curated, reaction-level pathway models for human biology. While both resources supply rich interaction evidence, neither organizes proteins into functional groups with verified co-membership. The CORUM database [17, 52] fills this gap by curating experimentally verified mammalian protein complexes exclusively from low-throughput experiments. By providing verified group membership, curated interaction maps serve as a high confidence reference for mechanistic relationships. These resources, however, cover only a small, well-studied fraction of the interactome and are costly to extend, motivating computationally derived module structures that scale across the entire genome.

Genome-Wide Module Discovery with MENTOR

A complementary approach to curated databases is deriving mechanistic ground truth directly from network structure. MENTOR (Multiplex Embedding of Networks for Team-Based Omics Research) [50] computes random-walk-restart (RWR) embeddings for every gene across the layers of a multiplex, converts the embeddings into pairwise distances, and applies hierarchical clustering to produce a genome-wide dendrogram in which leaves are genes and internal nodes represent successively coarser functional

groups. Selecting subtrees from this dendrogram yields a module whose granularity is controlled by the merge height at which the cut is taken, producing a nested family of modules that span the entire gene universe of the underlying network. Unlike curated databases, MENTOR provides module structure at genome scale and for arbitrary multiplexes, including tissue- and context-specific networks, trading literature-based verification for broad coverage and mechanistic coherence.

Gene-Centric Module Discovery with RWR-LOE

While MENTOR produces modules through top-down hierarchical clustering of the full gene universe, a complementary bottom-up approach derives modules for each individual gene. Applying RWR-LOE [24] to a single seed gene g yields a ranked list of all other genes, ordered by propagation proximity to the seed. RWR-LOE walks the entire multiplex, treating each layer as an independent line of evidence, so that final rankings reflect the combined network structure rather than a single interaction type. Rather than imposing a fixed neighborhood size, RWR++ elbow filtering selects the cutoff at the bend in the score-versus-rank curve for each seed. Retaining the seed and all genes before this adaptive cutoff yields a gene-centric module C_g^{loe}, producing one module per gene in the network. RWR-LOE modules provide a complementary, bottom-up source of mechanistic ground truth compared to hierarchical MENTOR modules.

Network Construction and Exploration on Multiplexes

Because biological systems comprise many complementary interaction modalities, integrating them without losing information motivated the development of the multiplex network formalism [31, 14], which represents the same node set across multiple edge layers of differing types. Multiplex networks support propagation methods such as random walk with restart (RWR) [25, 13], community detection methods, and learned graph models for tasks such as node classification, link prediction, and disease gene prioritization. In this proposal, these methods primarily motivate the graph operators available to the agent, while MENTOR and RWR-LOE define the module targets used for training and evaluation.

0.2.2 Tool-Using Language Models for Scientific Reasoning

Transformer Reasoning with Tool Use

In addition to deep learning's growing utility for biological discovery, neural language models have shown promise for complex reasoning tasks, especially when grounded with tool use. [56] demonstrated that prompting a model to show its reasoning steps before answering questions led to improved multi-step task performance in a few-shot setting, and [26] extended this result to show its effectiveness in a zero-shot scenario.

Furthermore, adding self-consistency mechanisms [54] and branched reasoning paths with lookahead and backtracking [60] have enhanced model reasoning capabilities.

Despite this progress in model reasoning, probabilistic decoding and training data cutoffs still leave language models prone to errors, including hallucination. In turn, equipping models with the ability to use external tools during training or inference provides a layer of deterministic verification and enables interaction with the outside world. Toolformer [43] showed that self-supervised fine-tuning of tool calls yielded better performance with lower hallucination across several benchmarks, and Gorilla [37] extended this paradigm to handle large API vocabularies with retrieval. ReAct [59] showed that combining reasoning techniques with tool-based actions improved complex reasoning performance, and PAL [15] converted reasoning into programs, showing improved performance on computational tasks. These works demonstrate the power of combined reasoning and tool use for complex, multi-step problem solving, but their applications to scientific reasoning remain underexplored.

Agentic Architectures with Verification

While reasoning and tool use techniques improve task solving capability, they lack an inherent verification mechanism, which is crucial for making factual scientific claims. Several works have explored self-verification through various methods. Self-Refine [28] and Reflexion [47] both define an iterative reflection loop without weight updates to improve answer quality. Self-Refine uses a single model to generate, critique, and refine its own output, while Reflexion utilizes sequential refinement, storing reflections in a persistent verbal memory. Self-RAG [2] fine-tunes a language model to emit reflection tokens that govern when to retrieve passages and how to assess their relevance and support. Eubiota [27] extends the self-verification process to multi-agent systems, using a dedicated verifier with separate weights to check the groundedness and factuality of other agent outputs. These approaches establish self-verification as a core component of agentic systems, but none combine a shared-weight actor-verifier paradigm with reinforcement learning over deterministic, structured state rewards, which we develop in Section **??**.

Agentic Gene Set Interpretation

LLM-based gene set interpretation has progressed from precomputed embeddings to fully agentic systems. GenePT [9] employs GPT-4 to create gene-level and cell-level embeddings from natural language for use in downstream interpretation tasks. GeneAgent [55] utilizes a multi-step GPT-4 pipeline with self-verification and retrieval from biological databases to provide a human-readable interpretation for experimentally derived gene lists. MENTOR-IA [53] extends this paradigm by creating separate agents for mechanistic interpretation subtasks, such as literature search, enrichment analysis, network analysis, and verification, unifying these outputs into a single mechanistic summary. Eubiota [27] functions similarly to MENTOR-IA, but

with a focus on the human gut microbiome. The authors partially train a planning agent with GRPO while keeping proprietary model-powered subagents frozen. While these works share MENTOR-RL's agentic framing for mechanistic interpretation, none implements a fully trainable, end-to-end policy over verifiable rewards, which we introduce in Section **??**.

0.2.3 Reinforcement Learning for Long-Horizon Tool-Use

Preference Optimization

Christiano et al. introduced deep reinforcement learning from human feedback (RLHF) [10], replacing hand-specified reward functions with a value model trained on pairwise human preferences. This framework was extended by InstructGPT [36], which applied RLHF to align large language model outputs with human preferences. Direct preference optimization [40] built on this work by deriving a mathematically equivalent objective that operates directly on preference pairs, only using the policy and a reference while foregoing the reward model. Azar et al. unified RLHF and DPO under ΨPO, a general preference optimization framework, and introduced IPO to address DPO's tendency to overfit when observed preferences are deterministic or near-deterministic [4]. Hong et al. proposed ORPO, a single-stage alternative that merges preference alignment and SFT via an odds-ratio penalty, removing the need for a separate alignment stage [20]. While these methods carry varying tradeoffs, we elect to train MENTOR-RL with DPO because our deterministic runtime supports scalable preference pair generation and the resulting checkpoint serves as a preference-aligned reference policy in the GRPO stage.

Policy Gradient and Group-Relative Methods

While preference optimization aligns LLMs to pairwise judgments, policy gradient methods optimize directly against scalar rewards over rollouts, supporting online learning across a wide range of tasks. Proximal policy optimization (PPO) [44] was introduced as a simple, efficient, and scalable alternative to TRPO [45], using a clipped probability ratio term to form a pessimistic surrogate of performance, preventing excessively large policy updates. In RLHF [36], this is paired with a token-wise KL penalty against a reference policy to anchor the model to its supervised initialization. Group relative policy optimization (GRPO) [46] builds on PPO, computing the advantage term by measuring the deviation of outputs from a group mean rather than using a trained value model. Removing the critic network roughly halves the trainable parameters relative to PPO, making large-scale training more feasible. DeepSeek-R1 [18] demonstrated that GRPO with verifiable rewards can elicit emergent long-horizon reasoning behaviors at frontier scale. Having no learned critic, group-normalized advantages, and compatibility with deterministic verifiable rewards makes GRPO conducive to MENTOR-RL, where the reward is a structured, schema-grounded signal over multi-hop biological graph exploration.

RL for Tool-Augmented Agents

In addition to its usefulness for general LLM training, RL and adjacent post-training methods have been extensively used to train LLM agents in tool-use scenarios. WebGPT [32] used a fine-tuning, PPO, and rejection sampling pipeline to train a language agent for browser use and citation-backed question answering. ReST^{EM} [49] frames self-training from synthetic data as expectation maximization, demonstrating that fine-tuning on model-generated, filtered data can outperform fine-tuning on human-labeled datasets in verifiable reward settings. Toolformer [43] demonstrated that LLMs can use external tools by sampling candidate API calls during pretraining and retaining those whose outputs reduce the loss on subsequent tokens. Building on this self-supervised paradigm, ToolLLM [39] trained a language model for multi-step, multi-tool question answering on over 16,000 APIs with imitation learning. Agent-Q [38] utilizes Monte Carlo tree search and process supervision to construct model trajectories, which are used to optimize an agent via DPO. These works provide methodological precedent for MENTOR-RL; however, we utilize on-policy trajectory generation and deterministic rewards grounded in network-derived targets as a source of truth.

0.3 Proposed Methodology

Our methodology has five components. Sections **??** and **??** formalize the iterative exploration loop and the mechanistic interpretation task. Sections **??** and **??** describe dataset construction and environment design. Sections **??**, **??**, and **??** define the training pipeline, reward functions, and curriculum. Finally, Section **??** describes evaluation and benchmarking.

0.3.1 Agent Loop Formulation

Loop Formalization

We model mechanistic graph exploration as a sequential decision process executed by a single policy model with an internal self-verification step. The loop begins when the user provides a query, q, and optional evidence e^{user}, which may include text context, seed genes, or a multiplex network. These inputs initialize the working interpretation I_0 and the structured state S_0.

For each decision point $t < T_{\max}$, the current decision context is

$$D_t = (q, e^{\text{user}}, I_t, S_t).$$

Given D_t, the agent emits an action consisting of a textual reasoning step r_t and an optional tool action a_t. The reasoning step specifies the current subgoal or interpretation update, while the tool action specifies a runtime-executable call. If

a tool action is emitted, it is executed by a deterministic runtime ε; otherwise, the observation is empty.

After the action is produced and any deterministic tool calls are executed, the same policy enters a verification mode. Conditioned on the query, user-provided evidence, the current interpretation, the current structured state, and the current reasoning/action outputs, the verifier updates the interpretation and structured state, yielding (I_{t+1}, S_{t+1}). It also emits a continuation decision,

$$z_t \in \{\texttt{continue}, \texttt{revise}, \texttt{stop}\},$$

which determines whether the current interpretation should be extended, revised, or terminated. The decision `revise` indicates that the next iteration proceeds by correcting the interpretation and structured state rather than simply extending them.

The process repeats until either $z_t = \texttt{stop}$ or the maximum decision budget $T_{\max}$ is reached. At termination, I_T and S_T are returned as outputs. I_T provides a human-readable answer to the query with respect to the accumulated evidence, while S_T stores the corresponding verifiable outputs for reward computation and provenance preservation.

We formalize one iteration of the agent loop as

$$r_t, a_t \sim \pi_\theta^{\text{act}}(\cdot \mid D_t), \qquad o_t = \varepsilon(a_t), \qquad I_{t+1}, S_{t+1}, z_t \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where r_t is the agent's textual reasoning or subgoal, a_t is the optional tool action, and o_t is the observation returned by the deterministic runtime ε, with $\varepsilon(\varnothing) = \varnothing$ by convention. The policies π_θ^{act} and $\pi_\theta^{\text{verify}}$ denote two prompting modes of the same underlying model, sharing weights so that biological vocabulary and tool-use competence learned in one mode transfers to the other. We treat the risk that the verifier collapses as a monitoring concern during training, discussed in Section **??**. At each decision point, the pair (I_t, S_t) represents the current mechanistic hypothesis, with I_t serving as a human-readable view and S_t as a machine-readable view of the same underlying state. The full agent loop algorithm is detailed in Algorithm **??**.

Working Interpretation

The working interpretation I_t is the human-readable form of the current mechanistic hypothesis and contains:

1. the current mechanistic claim,
2. the main evidence supporting that claim,
3. any uncertainty, conflict, or alternative explanation,
4. the next question or subgoal the model is trying to resolve.

Algorithm 1 The MENTOR-RL agent loop

Require: query q, user evidence e^{user}, maximum decision budget $T_{\max}$, deterministic runtime ε

Initialize working interpretation I_0 and structured state S_0 from (q, e^{user})
$T \leftarrow 0$
for $t = 0, 1, \ldots, T_{\max} - 1$ **do**
    $D_t \leftarrow (q, e^{\text{user}}, I_t, S_t)$
    $(r_t, a_t) \sim \pi_\theta^{\text{act}}(\cdot \mid D_t)$
    if $a_t \neq \varnothing$ **then**
        $o_t \leftarrow \varepsilon(a_t)$
    else
        $o_t \leftarrow \varnothing$
    end if
    $(I_{t+1}, S_{t+1}, z_t) \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t)$
    $T \leftarrow t + 1$
    if $z_t =$ `stop` **then**
        break
    end if
end for
return final interpretation I_T and structured state S_T

Structured State

The structured state S_t stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e^{user}),
2. the current relationship status (unknown, partially observed group, validated group, multiple groups, or insufficient support) (rel_t),
3. the current predicted group or groups (G_t),
4. the evidence and provenance collected so far (E_t),
5. the current predicted mechanistic labels, each tied to the evidence handles that support it,
6. the tool-use counters recording total and invalid tool calls (n_{total}, n_{inv}),
7. the remaining budget and continuation state, and, at termination, the recorded termination signal $((T_{\max} - T), z_t)$.

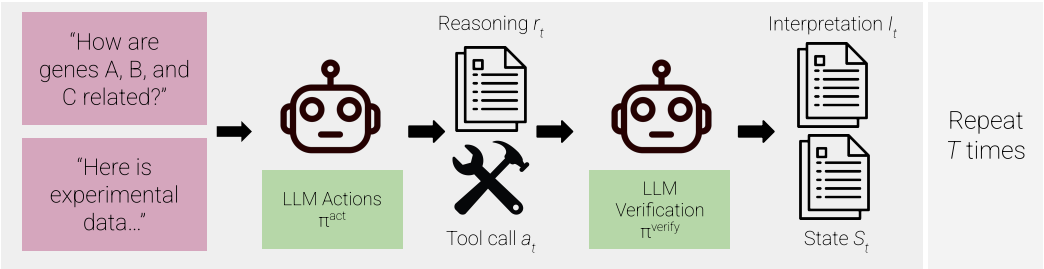

Figure 1: The MENTOR-RL agent loop.

0.3.2 Task Outputs and Mechanistic Targets

Final Interpretation

The final outputs of the model are the terminal interpretation I_T and the terminal structured state S_T. Together, these outputs must support four cases: completion of a partially observed mechanistic group, explanation of an already coherent group, refinement of a noisy group, and recognition that no single shared mechanism is supported. The final interpretation states which case is supported by the evidence, summarizes the main evidence for that claim, and links the evidence to the claim through reasoning traces and tool calls. It also records uncertainty or abstention when evidence is incomplete. If the loop terminates with $z_t = \texttt{stop}$, the next subgoal field is left blank. If the loop terminates because the decision budget $T_{\max}$ is exhausted, the interpretation may include a recommended next subgoal to guide further exploration.

Final Structured State

The final structured state is a schema-enforced JSON object. It contains the same components defined in Section **??**, but with finalized values for relationship status, predicted groups, collected evidence, and mechanistic labels. These values are used for reward evaluation and provenance tracking. The final structured state also records whether termination occurred because the model emitted $z_t = \texttt{stop}$ or because the budget $T_{\max}$ was exhausted. If the budget is exhausted, the remaining budget is 0; if the model stops early, the remaining budget is $T_{\max} - T$.

Supported Task Cases

We support four output cases during training to reflect the variability of experimental evidence that users may provide:

1. partial group completion,
2. mechanism explanation for an already related group,
3. group refinement from a noisy module,
4. no single shared mechanism.

Partial group completion occurs when the user provides a subset of related genes; the model expands this subset into a larger mechanistically related group and explains it. Mechanism explanation occurs when the user provides a complete module; the model explains the mechanism without unnecessary expansion. Group refinement occurs when the user provides a known module with several unrelated noise genes; the agent refines the group so that it contains only mechanistically related genes. Finally, in the no shared mechanism case, the user provides genes that do not support one coherent mechanism, and the model either splits them into multiple groups or returns insufficient support.

Targets for Training and Evaluation

We define two complementary target types for training and evaluation. **Module membership targets** are the hidden gene sets drawn from either MENTOR-derived dendrogram modules or elbow-filtered RWR-LOE modules (Sections **??** and **??**). These targets are only used for scoring: the agent never observes hidden module members and must recover them through exploration. These modules supply the supervised signal for set-level recovery, refinement, and abstention.

Because dendrogram modules are derived computationally from multiplex structure, they offer genome-wide coverage, but often without per-entry experimental annotations. **Mechanistic evidence targets** therefore score the quality of a final explanation against the evidence retrieved during exploration, such as enrichment, annotation, and graph support. This allows interpretations to be rewarded for being

grounded in evidence rather than simply matching a pre-defined annotation, which encourages the exploration of novel biology beyond existing databases.

0.3.3 Data, Resources, and Environment

Our dataset is constructed from a combination of existing internal tool infrastructure, open-source biological resources, and graph exploration environments.

Warm-Start SFT Dataset

We first construct a warm-start SFT subset from the MENTOR-IA tool stack [53], including gene metadata, gene ontology (GO) annotations, pathway membership, and literature evidence. These data are retrieved via MyGene.info [58] and Europe PMC [12]. This warm-start subset covers four task families:

1. entity normalization, such as resolving gene names and synonyms to canonical identifiers,
2. annotation retrieval, such as returning GO terms, localization, and pathway information for a gene or gene set,
3. literature-grounded extraction, such as identifying relevant genes, pathways, or mechanisms from retrieved abstracts, and
4. evidence-to-finding generation, which converts retrieved tool outputs into short, grounded mechanistic statements.

We include tool-enabled (open-book) and tool-disabled (closed-book) questions, with open-book examples emphasized to promote schema-valid tool use and evidence grounding, and closed-book examples used to strengthen biological vocabulary and lightweight recall. To make this stage reproducible, API responses and database versions will be cached during dataset construction.

To make the warm-start SFT stage concrete, we define a core tool vocabulary derived from the existing MENTOR-IA infrastructure [53] in Table **??**. Early SFT emphasizes schema-valid biological annotation and enrichment calls, while graph-exploration and RWR++ operators are introduced during DPO and online RL.

MENTOR-Derived Training Corpus

After constructing the warm-start SFT dataset, we create two complementary module corpora that together supply the hidden targets for trajectory generation: a MENTOR-derived corpus (this section) and an RWR-LOE corpus (Section **??**). We first parse the genome-wide MENTOR dendrogram into modules, where each module is the gene set of a dendrogram subtree. We partition the modules into train, validation, and test splits to prevent leakage across stages.

Table 1: Tool vocabulary for the MENTOR-RL agent. Related RWR++ calls are grouped by function while preserving the model-facing tool names.

Tool(s)	**Purpose**	**Primary inputs**	**Stage**
`query_mygene`	Gene metadata, GO terms, pathways, and summaries	`query`; requested `fields`	SFT
`enrich_gene_set`	Functional enrichment over a candidate gene set	`genes`; `background`; `top_k`; threshold	SFT, DPO, RL
`get_neighbors`; `induce_subgraph`	Local native graph and induced-subgraph evidence	gene or gene-set identifiers; layers	DPO, RL
`rwr`; `rwr_loe`; `shortest_paths`	RWR++ expansion, leave-one-out relatedness, and path evidence	seed, query, source, or target genes; layers; `top_k`	DPO, RL
`get_rank`; `get_distance`; `get_spearman`; `get_pearson`; `get_dot_similarity`	Rank, distance, and vector-similarity comparisons	paired genes; metric; layers	DPO, RL
`get_rank_vector_summary`; `get_encoding_summary`	Compact rank-vector and encoding summaries	seed or query genes; layers; `top_k`	DPO, RL
`get_gene_layers`; `get_nodes_by_layer`; `get_layer_stats`; `get_path_layer_counts`; `get_component_summary`	Layer membership, layer statistics, path support, and components	genes or layers, depending on query	DPO, RL
`get_seed_essentiality`; `get_layer_ablation`; `get_node_perturbation`	Stability, ablation, and perturbation analyses	seed or perturbation genes; layers; `top_k`	DPO, RL

Algorithm 2 Creation of the MENTOR-derived module training corpus

Require: dendrogram D, multiplex M, difficulty map δ

Parse the genome-wide MENTOR dendrogram D over multiplex M into modules $\{C_{\text{module}}\}$
Partition modules into train/validation/test splits at the module level
for each module C_{module} in split with difficulty map δ **do**
    $n \leftarrow |C_{\text{module}}|$
    // **explanation**
    emit ($C^{\text{in}} = C_{\text{module}},\ C = C_{\text{module}},$ `explanation`, $rel =$ `validated_group`)
    // **recovery**
    $k_{\text{drop}} \leftarrow \text{DROPCOUNT}(n, \delta_{\text{rec}})$; $G_{\text{drop}} \leftarrow \text{DETSELECT}(C_{\text{module}}, k_{\text{drop}})$
    emit ($C^{\text{in}} = C_{\text{module}} \backslash G_{\text{drop}},\ C = C_{\text{module}},$ `recovery`, $rel =$ `validated_group`)
    // **refinement**
    $k_{\text{add}} \leftarrow \text{NOISECOUNT}(n, \delta_{\text{ref}})$; $G_{\text{add}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, k_{\text{add}}, \delta_{\text{ref}})$
    emit ($C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}},\ C = C_{\text{module}},$ `refinement`, $rel =$ `validated_group`)
    // **none**
    $G_{\text{none}} \leftarrow \text{DENDRONEG}(C_{\text{module}}, n, \delta_{\text{none}},$ conflict_free)
    emit ($C^{\text{in}} = G_{\text{none}},\ C = \varnothing,$ `none`, $rel =$ `insufficient_support`)
end for
return all task instances

For each module C_{module} of size n, we deterministically materialize up to four task instances, one per case in Section **??**. This ensures that all four task types are represented for as many modules as possible rather than sampled independently. The `explanation` instance uses the full module as input, $C^{\text{in}} = C_{\text{module}}$. The `recovery` instance drops k_{drop} genes from the module, $C^{\text{in}} = C_{\text{module}} \backslash G_{\text{drop}}$, and the `refinement` instance adds k_{add} noise genes, $C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}}$. In both the recovery and refinement cases, the hidden target remains $C = C_{\text{module}}$. The number of genes added or dropped scales with the task difficulty and module size, so that harder instances perturb a larger fraction of the module. The `none` instance constructs a pseudo-module of unrelated genes as input with hidden target $C = \varnothing$.

Noise and `none` genes are drawn from difficulty-modulated *dendrogram distance bands*, which are defined by walking up the module's ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. Additionally, `none` genes are constrained to be conflict-free (i.e. they are not related to each other), so that sampled genes do not form a coherent module themselves. All selection is deterministic given a fixed seed, making the corpus reproducible. The resulting MENTOR-based corpus provides supervision for schema-valid tool use and for the terminal outputs of Section **??**.

Algorithm 3 Creation of the RWR-LOE module training corpus

Require: multiplex M, gene universe $\mathcal{G}$, elbow-filter parameters ϕ, difficulty map δ

Partition $\mathcal{G}$ into train/validation/test splits at the gene level
for each gene $g \in \mathcal{G}$ **do**
    Run `rwr_loe`(g, M) to obtain scores s_g and ranked list L_g
    $r_g^\star \leftarrow \textsc{ElbowCutoff}(L_g, s_g, \phi)$
    $E_g \leftarrow \{v \in L_g : \mathrm{rank}_g(v) \le r_g^\star\}$
    $C_g^{\mathrm{loe}} \leftarrow \{g\} \cup E_g, \quad n \leftarrow |C_g^{\mathrm{loe}}|$
    // **explanation**
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}},\ C = C_g^{\mathrm{loe}},\ \texttt{explanation},\ rel = \texttt{validated\_group})$
    // **recovery**
    $k_{\mathrm{drop}} \leftarrow \textsc{DropCount}(n, \delta_{\mathrm{rec}}); \quad G_{\mathrm{drop}} \leftarrow \textsc{DetSelect}(C_g^{\mathrm{loe}}, k_{\mathrm{drop}})$
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}} \setminus G_{\mathrm{drop}},\ C = C_g^{\mathrm{loe}},\ \texttt{recovery},\ rel = \texttt{validated\_group})$
    // **refinement**
    $k_{\mathrm{add}} \leftarrow \textsc{NoiseCount}(n, \delta_{\mathrm{ref}}); \quad G_{\mathrm{add}} \leftarrow \textsc{RankBandNeg}(L_g, r_g^\star, k_{\mathrm{add}}, \delta_{\mathrm{ref}})$
    emit $(C^{\mathrm{in}} = C_g^{\mathrm{loe}} \cup G_{\mathrm{add}},\ C = C_g^{\mathrm{loe}},\ \texttt{refinement},\ rel = \texttt{validated\_group})$
    // **none**
    $G_{\mathrm{none}} \leftarrow \textsc{RankBandNeg}(L_g, r_g^\star, n, \delta_{\mathrm{none}}, \mathrm{conflict\_free})$
    emit $(C^{\mathrm{in}} = G_{\mathrm{none}},\ C = \varnothing,\ \texttt{none},\ rel = \texttt{insufficient\_support})$
end for
return all task instances

RWR-LOE Module Corpus

In addition to the top-down, hierarchical MENTOR-derived module corpus, we create a corpus of RWR-LOE exploration across all genes in the multiplex to provide a bottom-up complement. For each gene g in the same multiplex universe as the MENTOR dendrogram, we run `rwr_loe`(g) to obtain a propagation score and rank for every other gene in the network. We then apply the RWR++ elbow filter to the score-versus-rank curve for that seed, yielding an adaptive cutoff rank $r_g^\star$. The LOE module is defined as $C_g^{\mathrm{loe}} = \{g\} \cup \{v : \mathrm{rank}_g(v) \le r_g^\star\}$, producing a single seed-specific module per gene without imposing a fixed module size.

For each module, we materialize the same four task instances as the MENTOR-derived corpus. For difficulty control, we substitute RWR-LOE rank bands for dendrogram distance bands: noise and `none` genes are drawn from ranks just beyond, moderately beyond, or far beyond the seed's elbow cutoff. We partition RWR-LOE modules into train, validation, and test splits independently of the MENTOR-derived modules, and module-size bins are assigned after construction from the realized values of $|C_g^{\mathrm{loe}}|$. The two corpora are complementary: MENTOR modules capture top-down, hierarchical functional organization, while RWR-LOE modules reflect bottom-up, seed-centered topology.

Graph Environment and Runtime

Each task instance is paired with a graph environment defined over the same canonical gene universe as the in-house brain multiplex, from which our module corpora are derived. The sampled input set C^{in} initializes the agent's graph anchors, while the hidden target C defines the supervisory signal.

To interface the graph environment with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [23]. Training, rollout, and optimization code are implemented in Python [42], while graph storage and graph operations are handled by the native runtime for efficient execution on large multiplexes. Multiplex networks are persisted as raw, CSR-style binary stores described by JSON manifests; all graph input and output is binary, with the manifest carrying the layer names, node, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [50] and MENTOR-IA [53] projects known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the agent during exploration.

All runtime operators return both graph objects and provenance metadata. In addition to nodes, edges, and scores, each returned object records the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers.

0.3.4 Trajectory Generation, Preference Mining, and Mechanistic Summary Construction

Overview

Using the module corpora and deterministic runtime defined in Sections **??**, **??**, and **??**, we generate shared-prefix trajectories for SFT and DPO. For each sampled task, the warm-start policy proposes multiple candidate next steps, deterministic execution produces candidate observations and next states. We employ structured branch scoring functions to rank these candidates, and the resulting rankings are converted into shared-prefix preference pairs. One branch per step is retained as the continuation of the logged trajectory, from which we derive per-step findings and a final mechanistic summary. The trajectory generation process is outlined in Algorithm **??**.

Algorithm 4 Trajectory Generation for SFT and DPO

Require: $\pi_\theta^{act}, \pi_\theta^{verify}$, runtime ε, $T_{\max}, n_{act}, n_{ver}, n_{cov}$, directives $\{d_i\}$, pair margin τ, selection tolerance ϵ.

Sample module task $(C^{\text{in}}, C, C_{\text{type}})$ and evidence mode ρ.

$q, e^{\text{user}} \sim Q_{\text{templates}}(\cdot \mid C^{\text{in}}, C, C_{\text{type}}, \rho)$.

$(I_0, S_0) \leftarrow \text{InitState}(q, e^{\text{user}}, C^{\text{in}})$.

$\tau^\star \leftarrow [\,], \quad \mathcal{P} \leftarrow [\,], \quad \mathcal{F} \leftarrow [\,], \quad T \leftarrow 0$

$z_{-1} \leftarrow \texttt{continue}$

for $t = 0, 1, \ldots, T_{\max} - 1$ **do**

  if $z_{t-1} = \texttt{stop}$ **or** $\text{budget}(S_t) \le 0$ **then break**

  end if

  $D_t \leftarrow (q, e^{user}, I_t, S_t), \quad \mathcal{C}_t \leftarrow [\,], \quad \mathcal{A} \leftarrow [\,]$

  for $i = 0, \ldots, n_{act} - 1$ **do**

    $(r_{t,i}, a_{t,i}) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i)$; append to $\mathcal{A}$

  end for

  if $C_{\text{type}} \in \{\texttt{recovery}, \texttt{refinement}\}$ **and** no $a \in \mathcal{A}$ is a tool call **then**

    for $i = 0, \ldots, n_{cov} - 1$ **do**

      $(r, a) \sim \pi_\theta^{act}(\cdot \mid D_t, d_i^{\text{cov}})$; append to $\mathcal{A}$

    end for

  end if

  for each $(r_{t,i}, a_{t,i}) \in \mathcal{A}$ **do**

    $o_{t,i} \leftarrow \varepsilon(a_{t,i})$ if $\text{ValidTool}(a_{t,i}, S_t)$ else $\varnothing$

    for $j = 0, \ldots, n_{ver} - 1$ **do**

      $(I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j}) \sim \pi_\theta^{verify}(\cdot \mid D_t, r_{t,i}, a_{t,i}, o_{t,i})$

      $s^{local}_{t,i,j} \leftarrow \text{LocalScore}(D_t, I'_{t,i,j}, S'_{t,i,j}, r_{t,i}, a_{t,i}, o_{t,i}, z'_{t,i,j}; C, C_{\text{type}})$

      Append $c_{t,i,j}$ to $\mathcal{C}_t$

    end for

  end for

  $\text{NormalizeScores}(\mathcal{C}_t)$

  $(i^\star, j^\star) \leftarrow \text{SelectBranch}(\mathcal{C}_t; \epsilon)$

  $\mathcal{P} \leftarrow \mathcal{P} \cup \text{MakePreferencePairs}(\mathcal{C}_t, c_{t,i^\star,j^\star}, \tau)$

  Append $(I_t, S_t, c_{t,i^\star,j^\star})$ to $\tau^\star$; $f_t \leftarrow \text{RenderFinding}(\cdot)$; append f_t to $\mathcal{F}$

  $(I_{t+1}, S_{t+1}) \leftarrow (I'_{t,i^\star,j^\star}, S'_{t,i^\star,j^\star}); \quad T \leftarrow t + 1$

end for

$m \leftarrow \text{RenderSummary}(S_T, \mathcal{F})$

return $\tau^\star, \mathcal{F}, m, \mathcal{P}$

Shared-prefix Trajectory Synthesis

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ given the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore genuinely distinct regions of the action space, we use verbalized sampling based on the task type. For example, recovery prompts bias candidates towards random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision, while insufficient evidence (`none`) prompts direct the model towards disconfirming graph evidence. When a sampling round for recovery or refinement tasks fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, which prevents the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy then samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. The branched sampling scheme is visualized in Figure **??**.

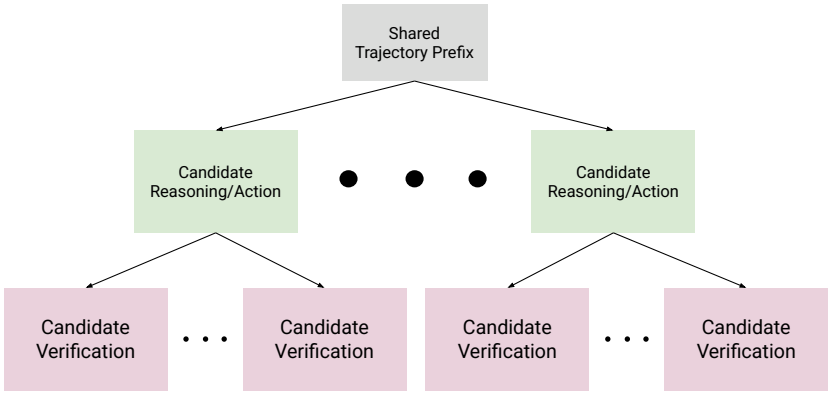

Figure 2: The shared prefix generation tree.

Initialization, Tool Validation, and Query Construction

We define InitState$(q, e^{\text{user}}, C^{\text{in}})$ as a deterministic function that constructs the initial interpretation I_0 and structured state S_0 from the query, user-provided evidence, and seed gene set. The working interpretation I_0 is populated from a fixed template: the mechanistic claim is empty, the evidence field records only the user-provided inputs, the uncertainty field is blank, and the next subgoal is set to the query q. The structured state S_0 is initialized as follows:

1. the user-provided anchors are set to (q, e^{user}),
2. the relationship status rel_0 is set to `unknown`,
3. the predicted group is initialized to $G_0 = C^{\text{in}}$,
4. the evidence log E_0 is empty,
5. the predicted mechanistic labels are empty,
6. the remaining budget is $T_{\max}$ and the continuation state is $z_0 =$ `continue`.

The agent does not observe the task type C_{type}, the true relationship status, or the hidden target C; it must infer the appropriate task case through exploration.

We define ValidTool$(a_{t,i}, S_t)$ as a deterministic function that checks whether a proposed tool action $a_{t,i}$ is executable given the current structured state S_t. Validation proceeds in two stages. First, the syntax check verifies that the tool name referenced in $a_{t,i}$ exists in the current tool vocabulary and that all arguments conform to the tool's input schema. Second, the semantic check verifies that basic conditions are satisfied with respect to the graph environment. For example, the function ensures that queried genes exist in the multiplex, requested layers are valid, and required fields in S_t are populated. If either check fails, ValidTool returns false and the observation $o_{t,i}$ is set to $\varnothing$. Invalid tool calls are counted toward n_{inv} in the efficiency penalty (Equation **??**).

At inference time, users provide a natural language query q alongside optional evidence e^{user}. Queries are unconstrained and may range from broad exploratory requests to specific hypothesis-driven questions. Evidence may include any combination of:

1. a seed gene list,
2. a multiplex graph or subgraph,
3. free-text context such as experimental notes or prior findings, and
4. structured annotations such as GO terms, pathway labels, or expression data from prior analysis.

During training, we simulate this variability by generating diverse queries and evidence configurations from module metadata. For each sampled task $(C^{\text{in}}, C, C_{\text{type}})$, we sample a query from task-type-specific deterministic templates and an evidence mode $\rho \in \{\texttt{minimal}, \texttt{graph}\}$ that controls which evidence types are provided to the agent. Table **??** defines representative query templates and compatible evidence modes for each task type. To promote phrasing diversity, we use an LLM to paraphrase each template query into natural language variants conditioned on the gene symbols of the sampled module. All generated queries and evidence configurations are cached during dataset construction for reproducibility.

Table 2: Representative query templates and compatible evidence modes by task type. Evidence modes: `minimal` (seed gene list) or `graph` (seed gene list + multiplex subgraph).

Task type	**Example query**	**Evidence modes**
`recovery`	"What other genes are functionally related to {seed genes}?"	minimal, graph
`refinement`	"Which of these genes do not belong together?"	minimal, graph
`explanation`	"What mechanism connects {seed genes}?"	minimal, graph
`none`	"Are these genes mechanistically related?"	minimal, graph

Branch Ranking, Preference Mining, and Selection

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden module targets. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches frequently tie or fall within a small margin of each other, continuation selection does not reduce to a strict $\arg\max$, but instead, within a tolerance margin ϵ of the best normalized score, ties are broken with task-specific heuristics such as prompted seed gene retention and tool-supported evidence, pushing the logged trajectory to favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section **??**.

0.3.5 Preference Scoring and Reward Design

Design Principles and Scoreable Objects

We define all training-time scores over structured state transitions, deterministic runtime metadata, and hidden module-derived targets rather than textual reasoning. This design choice keeps scoring reproducible, machine-parseable, and scalable, while avoiding dependence on human judges or semantic evaluation of natural-language

trajectories. We decompose all scores into four components: schema compliance, group membership, mechanistic quality, and efficiency.

Deterministic Local Branch Scoring for DPO

We define local branch scoring for DPO as

$$s_{t,i,j}^{local} = w_s\, s^{schema} + w_c\, \Delta s^{module} + w_m\, \Delta s^{mech} - w_e\, p^{eff} - p^{mismatch} \tag{1}$$

where Δs^{mech} is the evidence-grounded mechanistic quality delta. Additionally, mechanistic quality is clamped to be non-positive when the candidate degrades the predicted group relative to the task type (e.g. by adding noise genes during recovery or removing true members during refinement), and $p^{mismatch}$ penalizes an `explanation` candidate that reduces recovery of an already-complete module.

We define $s_{t,i,j}^{schema}$ as a binary indicator that equals 1 if the candidate's structured state and interpretation updates are schema valid, and 0 otherwise. Specifically, schema compliance is defined by a properly-formed JSON with all required fields present and tool calls referencing valid tool names and arguments. We formalize this term as

$$s_{t,i,j}^{schema} = \mathbf{1}(\text{SchemaValid}(c_{t,i,j})). \tag{2}$$

We use Δs_{module} to measure the change in module membership accuracy from step t to $t+1$. Because the runtime state may contain multiple predicted groups, we first score a state by the best-matched predicted group:

$$s^{module}(z; C, C_{\text{type}}) = \max_{G \in \mathcal{G}(z)} \left[\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)\right], \tag{3}$$

where $\mathcal{G}(z)$ denotes the set of predicted groups in state z, with an empty group used when no prediction is present. We then define the local module improvement as

$$\Delta s_{t,i,j}^{module} = s^{module}(z_{t+1,i,j}; C, C_{\text{type}}) - s^{module}(z_t; C, C_{\text{type}}). \tag{4}$$

where J, P, and R denote Jaccard, precision, and recall respectively, the weights α, β, γ are set per task type. Recovery upweights recall ($\alpha, \beta, \gamma = 0.2, 0.2, 0.6$) to reward found members, refinement upweights precision $(0.2, 0.6, 0.2)$ to penalize retained noise, and explanation weights them equally ($\frac{1}{3}$ each). This best-group formulation supports the multiple-groups state representation while keeping the hidden target single-module for scoring.

We define the mechanism quality of a state as an evidence-grounded score $m_t \in [0, 1]$. From the evidence log, we extract three support signals: enrichment support e_t, annotation support g_t (from `query_mygene`), and network support u_t, each $\in [0, 1]$. We define a consensus term $\kappa_t = \max(e_t, g_t)$, a cross-tool agreement term $\chi_t = \min(e_t, g_t)$, an evidence strength term $\sigma_t = \max(e_t,\ 0.75\, g_t,\ 0.5\, u_t)$, and an annotation-only strength term $\sigma_t^{\text{ann}} = \max(e_t,\ 0.75\, g_t)$. We additionally compute a grounding term γ_t (claim entities are supported by observed evidence), a specificity

term ψ_t (vague or generic claims are penalized), a stability term b_t (enrichment consistency across query subsets), and a discounted network-agreement term $\hat{u}_t = u_t$ if $\kappa_t > 0$ and $0.35\,u_t$ otherwise. An abstention term α_t rewards withholding a claim under weak support: $\alpha_t = 1 - \sigma_t$ when the state abstains (relationship status is `insufficient_support` or an abstaining claim), 0 when an unsupported claim is asserted ($\sigma_t \leq 0.05$), and 0.5 otherwise.

The score takes one of two branches depending on whether the state abstains:

$$m_t = \begin{cases} 0.60\,\alpha_t + 0.20\,\hat{u}_t + 0.20\,\gamma_t & \text{(abstaining)} \\ 0.25\,\gamma_t + 0.25\,\kappa_t + 0.18\,\psi_t + 0.12\,\hat{u}_t + 0.10\,b_t + 0.10\,\chi_t & \text{(asserting)} \end{cases}$$

In the asserting branch, the score is multiplied by 0.35 when a claim or labels are present but unsupported ($\sigma_t \leq 0.05$). In the abstaining branch m_t is forced to 0 for any non-`none` task whose annotation evidence is absent ($\sigma_t^{\text{ann}} \leq 0.05$), preventing weak-evidence abstention from being rewarded on tasks that should produce a grounded group. The per-step mechanism reward is the change $\Delta s^{mech}_{t,i,j} = m_{t+1,i,j} - m_{t,i,j}$, set to 0 if no mechanistic claim has been made.

p_{eff} is a small penalty designed to discourage two behaviors: excessively long trajectories and invalid tool calling. Long trajectories are penalized via the term $t/T_{\max}$, while we count invalid tool calls (those that are schema-invalid, exact duplicates, or return empty observations) with n_{inv}, penalizing them proportionally to the total number of tool calls: (n_{inv}/n_{total}). We formalize this scoring term as

$$p^{eff}_{t,i,j} = \lambda_1(t/T_{\max}) + \lambda_2(n_{inv}/n_{total}). \tag{5}$$

For the `none` task type, where the hidden target is empty, the membership component is replaced by an abstention score

$$s^{none} = w_{rel}\,\mathbf{1}[rel = \texttt{insufficient\_support}] + w_{abs}\,\mathbf{1}[G = \varnothing],$$

with $w_{rel} = 0.6$ and $w_{abs} = 0.4$. This rewards the agent both for declaring no shared mechanism and for refusing to emit a spurious group.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s^T_{schema} + w^T_{abs} s^T_{module} + w_c^T \Delta s^T_{module} + w^T_{mabs} s^T_{mech} + w^T_m \Delta s^T_{mech} - w_e^T p^T_{eff}. \tag{6}$$

While all terms included in local scoring appear in the terminal reward, they are modified to capture global behavior. Additionally, two new rewards are included

to capture absolute recovery. Absolute terms appear in the terminal score but not the local score. Because local scoring is evaluated on examples that share the same starting state S_t, absolute levels cancel, and only relative metrics matter. Conversely, the terminal reward is evaluating complete trajectories, which may vary in starting difficulty and length, so absolute metrics capture global problem-solving ability.

We define the absolute module membership score as the composite Jaccard, precision, and recall evaluated on the final predicted group G_T against the hidden target C:

$$s^T_{module} = \alpha J(G_T, C) + \beta P(G_T, C) + \gamma R(G_T, C), \tag{7}$$

using the same weighting coefficients α, β, γ defined in Equation **??**. The cumulative module delta is the difference between the final and initial composite scores:

$$\Delta s^T_{module} = s^T_{module} - s^0_{module}, \tag{8}$$

where $s^0_{module} = \alpha J(G_0, C) + \beta P(G_0, C) + \gamma R(G_0, C)$ is the composite score at initialization.

The absolute mechanism score is the evidence-grounded mechanism quality m_T of the final state:

$$s^T_{mech} = m_T. \tag{9}$$

The cumulative mechanistic delta is the corresponding difference:

$$\Delta s^T_{mech} = m_T - m_0. \tag{10}$$

If no mechanistic claim has been made at termination, we set $s^T_{mech} = 0$ and $\Delta s^T_{mech} = 0$.

The terminal schema score checks that the final structured state is schema-valid and that the termination mode is properly recorded:

$$s^T_{schema} = \mathbf{1}(\text{SchemaValid}(S_T) \wedge \text{TerminationRecorded}(S_T)). \tag{11}$$

Finally, the terminal efficiency penalty aggregates trajectory length and invalid tool use across all steps:

$$p^T_{eff} = \lambda_1 \left(\frac{T}{T_{\max}} \right) + \lambda_2 \left(\frac{\sum_{t=0}^{T-1} n^t_{inv}}{\sum_{t=0}^{T-1} n^t_{total}} \right). \tag{12}$$

Preference Pair Construction, Score Normalization, and Filtering

At each decision point t, we create DPO preference tuples of the form (x_t, c^+_t, c^-_t) from our candidate set $\mathcal{C}_t = \{c_{t,i,j}\}$. We define $x_t = (q, e^{\text{user}}, I_t, S_t)$ as the shared-prefix context and $c_{t,i,j}$ as a sample interpretation continuation from Algorithm **??**. Each continuation contains the model-generated reasoning step, tool action, verifier update, and continuation decision.

Before constructing pairs, we normalize scores to ensure that the selection margin has a consistent meaning across task types and trajectory lengths. We apply min-max normalization within each candidate pool $\mathcal{C}_t$, rescaling the composite local scores to $[0, 1]$. We denote the normalized score of candidate $c_{t,i,j}$ as $\bar{s}^{\text{local}}_{t,i,j}$.

To create paired examples, we set the preferred candidate c_t^+ to the branch selected as the trajectory continuation, $c_{t,i^\star,j^\star}$ (Section **??**), rather than to a strict arg max of the normalized score. Because selection resolves near-ties with task-specific heuristics within a tolerance ϵ, the preferred branch is the one judged best for the task, not merely the highest-scoring candidate. We then define a selection margin τ and drop all candidates for which

$$\bar{s}^{\text{local}}_{t,i^\star,j^\star} - \bar{s}^{\text{local}}_{t,i,j} < \tau,$$

where $(i^\star, j^\star)$ indexes the selected candidate. This prevents the construction of pairs that are near ties. The remaining candidates form the eligible dispreferred pool $\mathcal{C}_t^-$.

From $\mathcal{C}_t^-$, we draw dispreferred examples across three difficulty bins, $d \in \{\texttt{easy}, \texttt{medium}, \texttt{hard}\}$. We construct `easy` pairs by selecting the lowest-scoring candidate in $\mathcal{C}_t^-$, `medium` pairs by selecting the candidate with the median score, and `hard` pairs by selecting the highest-scoring candidate in $\mathcal{C}_t^-$.

We store each preference pair (x_t, c_t^+, c_t^-) along with its difficulty bin d, task type C_{type}, decision step t, raw and normalized scores, and the score margin in the preference corpus $\mathcal{P}$. After trajectory generation is complete, we rebalance $\mathcal{P}$ across task types and difficulty bins by downsampling overrepresented categories to ensure that the training distribution is not dominated by task types that produce disproportionately many informative pairs. The curriculum strategy described in Section **??** controls which difficulty bins are included at each training stage.

Mechanistic Rendering and Training Outputs

Each selected trajectory is logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These artifacts are then converted into SFT trajectory examples, evidence-to-finding records, and DPO preference pairs for the downstream training pipeline. Additionally, scored trajectories can be utilized for policy gradient optimization, such as PPO or GRPO.

0.3.6 Training Pipeline

We propose a training pipeline that closely aligns with standard model post-training pipelines. First, we utilize supervised fine-tuning for general biological knowledge and tool calling. We continue training with DPO on stepwise interpretation tree comparisons. Finally, we promote novel exploration over long-horizon tasks with policy gradient optimization.

Warm-Start SFT

We first train our policy model with warm-start SFT using the closed-book dataset outlined in Section **??**. We will train our policy model on OLCF's Frontier Supercomputer [3], utilizing slurm [61] for job scheduling, huggingface and TRL [57] for weight updating, DeepSpeed ZeRO-3 for model parallelism [41], Low-Rank Adaptation for parameter-efficient fine-tuning [21], Weights and Biases [8] for tracking convergence, and gpt-oss-120b [34] as our base policy. We will utilize early stopping checkpointing to prevent model overtraining.

Tool-Call and Evidence-to-Finding SFT

We continue the supervised fine-tuning stage by adding open-book tool calling examples and evidence-to-finding examples to our data mixture. We initialize our policy from the warm-start SFT checkpoint with the lowest loss, and we utilize the same pipeline for model training.

DPO Training

After training our model via SFT, we will utilize its learned domain knowledge and tool calling capabilities to perform DPO training. We will utilize examples generated with the trajectory construction methods specified in **??** to create paired preference data. Branch continuations will be scored using the functions defined in Section **??**. At each decision point, the branch selected as the trajectory continuation serves as the preferred candidate c_t^+, while dispreferred candidates c_t^- are drawn from the eligible dispreferred pool across the `easy`, `medium`, and `hard` difficulty bins. Model weights will be optimized using the DPO objective function [40]:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Additionally, during training, we will generate trajectories with new, trained model checkpoints to improve our model's generated preference data. This will allow us to continue making model improvements even after scoring is optimized on our initial dataset.

Online RL for Long-Horizon Exploration

After training with DPO, we will implement a final GRPO stage for long-horizon exploration. Unlike DPO, GRPO requires no annotated trajectories; rather, optimization is done by calculating the advantage of an output relative to the rollout group $\{o_1, o_2, \ldots, o_G\}$ using the function

$$\mathcal{J}_{GRPO} = \mathbb{E}[q \sim P(Q), \{y_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(Y \mid q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{\mid y_i \mid}\sum_{t=1}^{|y_i|}\left\{\min\Big[\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}\hat{A}_{i,t}, \text{clip}\Big(\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}, 1-\varepsilon, 1+\varepsilon\Big)\hat{A}_{i,t}\Big] - \beta\mathbb{D}_{KL}[\pi_\theta \mid\mid \pi_{ref}]\right\},$$

where

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{R_i^T - \text{mean}(R^T)}{\text{std}(R^T)}$$

By foregoing structured, annotated preference pairs, GRPO promotes novel exploration for problem solving. We aim to utilize this policy gradient method to prevent mode collapse in exploration trajectories and reinforce full coverage of the exploration space. The reference policy π_{ref} is fixed at the DPO checkpoint that initializes GRPO and is not updated during online rollouts, so the KL term anchors exploration to a stable reference rather than drifting with the trained policy.

0.3.7 Curriculum and Optimization Strategy

We will employ a training curriculum throughout the optimization process, scaling problem difficulty with model performance.

During the SFT stage, we scale difficulty by moving from closed-book biological questions to open-book biological questions that require tool use. This promotes learning biological understanding first, and then moving to structured, schema-respecting tool calling tasks second.

After training a joint closed- and open-book SFT checkpoint, we scale difficulty for DPO pairs by training on increasingly difficult examples, based on difficulty bins defined in Section **??**. We begin DPO training by limiting preference pairs to $d =$ `easy` examples, then adding `medium` and `hard` examples as training saturates for each difficulty. After DPO training converges on `hard` preference pairs, we transition to online RL to promote exploration beyond the behaviors captured in logged trajectory outputs.

We scale long-horizon reinforcement learning difficulty with several strategies. First, we modulate the trajectory length $T_{\max}$, starting with shorter exploration budgets and gradually increasing exploration length for problems that require multi-hop exploration. Additionally, we modulate realized module size, beginning training on small modules and shifting to larger ones as rewards saturate. Finally, we scale the per-task difficulty level, which jointly controls the number of dropped and added genes and the source-specific distractor band from which noise and `none` genes are drawn:

dendrogram-distance bands for MENTOR-derived modules (Section **??**) and post-elbow rank bands for RWR-LOE modules (Section **??**). Higher difficulties remove or add more genes and draw distractors from nearer, more easily confused regions of the corresponding module source, making recovery and refinement harder. At each stage, training batches are sampled from both the MENTOR-derived and RWR-LOE module pools and are rebalanced by task type and difficulty bin to prevent either corpus from dominating training.

0.3.8 Evaluation and Benchmarks

Module Holdout Evaluation

We will use several metrics to evaluate the effectiveness of our model. Each metric will be calculated by parsing the model's terminal structured state S_T, allowing for scalable evaluation of our checkpoints. All evaluations will be conducted on a held out set of MENTOR- and RWR-LOE-derived modules from the brain multiplex.

We will report four metrics for evaluation: Jaccard index J, precision P, recall R, and F_1 score on the predicted module C^{pred} vs. the hidden module target C. Jaccard index is defined as

$$J(C^{pred}, C) = \frac{\mid C^{pred} \cap C \mid}{\mid C^{pred} \cup C \mid},$$

penalizing both missing and extra entities. Precision is defined as

$$P = \frac{TP}{TP + FP},$$

measuring the proportion of true positive entities compared to all predicted positives. Recall is defined as

$$R = \frac{TP}{TP + FN},$$

measuring the proportion of true positive predictions to the sum of true positives and false negatives. The F1 score is defined as the harmonic mean of precision and recall,

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}},$$

providing a balanced view of precision and recall to measure general performance. In addition to these module recovery metrics, we will measure mechanistic grounding quality, defined in Equation **??**, to assess whether mechanistic claims are supported by retrieved evidence.

We will report the weighted composite reward R_T, defined in Section **??**, to directly compare training reward magnitude on validation and holdout splits. If the task type is `none`, meaning $C = \varnothing$, we will parse the S_T for the predicted relationship rel_T. Along with computing global accuracy, we will distinguish between both error

directions, analyzing when the model claims no mechanism exists for truly related gene sets and when the model fabricates a mechanism for truly unrelated entities.

Furthermore, we will report efficiency metrics, such as mean trajectory length, budget utilization, and invalid tool call rate. This allows for comparison between holdout efficiency and validation efficiency metrics computed in Equation **??**.

In addition to general evaluation, we will stratify our evaluation across several axes: module size, noise, task type, and module source. We will divide module size and noise into three discrete bins each for comparative evaluation. Reporting stratified metrics allows us to analyze how MENTOR-RL performs as task difficulty increases and task type changes, and comparing metrics by module source exposes whether the model generalizes across module construction methods or exhibits biases toward one of the two module types.

Novel-Domain Evaluation

Because the holdout split above is drawn from the same brain multiplex used for training, it measures in-distribution generalization. To assess out-of-distribution performance, we additionally evaluate on MENTOR-derived modules from independently published multiplex studies the model was not trained on, such as networks built for opioid use disorder and cardiac studies. These modules are produced by the same MENTOR dendrogram procedure (Section **??**) over different tissue- and disease-specific multiplexes, so they share the module-recovery scoring methodology while presenting genuinely novel network context. We report the same metrics defined in Section **??** and expect a performance gap relative to the brain-multiplex holdout, reflecting distribution shift. Because these modules derive from real experimental networks with expert- and algorithm-verified structure, they let us characterize the strengths and limitations of MENTOR-RL on real-world, out-of-distribution data.

Blind Comparative Evaluation

In conjunction with empirical validation, which measures model accuracy, we will conduct blind comparative evaluations to measure expert domain scientists' model preferences. First, we will perform an inter-model preference test, supplying interpretation outputs from MENTOR-RL, GPT-5 [48], gpt-oss-120b [34], LLaMA 4 [1], Kimi K2 [51], and Nemotron 3 Super [33]. We will provide between 5 and 15 domain experts with paired samples containing MENTOR-RL outputs and outputs from another randomly selected model. We will then ask scientists to indicate their preferred interpretation, and with this information, we will calculate the proportion of MENTOR-RL outputs that are preferred compared to other models. To support interpretation of these proportions, we will overlap a subset of evaluation pairs across raters and report inter-expert rating reliability.

After conducting inter-model preference testing, we will perform intra-model preference testing to compare preferences for different model checkpoints. Using the

same protocol described for inter-model comparative evaluation, we will compare the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. Blind comparative testing will allow us to evaluate model characteristics such as style, tone, and concision that are not easily evaluable with empirical metrics.

Efficiency and Ablations

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section **??**.

We will ablate phases of our training pipeline to confirm that each stage improves over the previous checkpoints. First, we will test our SFT-only checkpoint compared to the SFT- and DPO-trained checkpoint to ensure that preference optimization yields interpretation gains. Next, we will compare the DPO checkpoint to the full training pipeline to determine whether long-horizon RL training leads to improvement over local preference training. Finally, we will examine the effects of training with only closed-book SFT data vs. a mixed closed- and open-book training set to evaluate whether tool-calling examples in SFT lead to better DPO optimization.

Along with testing various training stages, we will ablate architectural and reward decisions. First, we will ablate the policy's verifier mode, removing it to test whether self-verification is useful for producing better results. Additionally, we will remove the efficiency penalty p^{eff} from the reward to test whether MENTOR-RL can learn reasonable trajectory lengths without regularization. Finally, we will ablate difficulty bins in our training curriculum, training on all difficulty bins simultaneously rather than staging them progressively. This ablation will allow us to examine whether the model can learn from hard examples without curriculum staging.

0.4 Expected Results and Deliverables

0.4.1 Expected Empirical Results

We expect MENTOR-RL to recover held-out MENTOR-derived and RWR-LOE modules with a composite F_1 competitive with frontier models at substantially lower per-query inference cost while improving monotonically across the training pipeline. Each of the SFT, DPO, and GRPO checkpoints should outperform the preceding one on module recovery and mechanistic grounding. Under the stratified evaluation defined in Section **??**, we expect the largest gains on `recovery` and `refinement` tasks, where deterministic graph operators provide dense supervisory signals, and smaller but nontrivial gains on `explanation` and `none` tasks, where reward is driven by mechanistic grounding and abstention rather than set-level recovery. On the novel-domain evaluation against experimentally-derived MENTOR modules, we expect a performance gap relative to brain multiplex holdout, reflecting distributional shift, but with the gap narrowing as curriculum difficulty and noise levels increase during

training. We expect comparable F_1 on held-out RWR-LOE modules relative to MENTOR-derived modules given the shared task structure, with any gap reflecting differences in module coherence between the two construction methods.

0.4.2 Expected Behavioral Results

Beyond empirical accuracy, we expect three behavioral outcomes. First, the act-verify loop should reduce schema-invalid tool calls and unsupported mechanistic claims relative to a verifier-ablated baseline, measured by invalid tool call rate and ungrounded-claim rate on the holdout split. Second, the efficiency penalty should produce shorter average trajectories without loss of recovery accuracy relative to a penalty-ablated baseline. Third, blind comparative evaluation should show that expert scientists prefer MENTOR-RL outputs to base-policy outputs at a significantly higher rate, and preferences relative to frontier models should improve across successive training stages.

0.4.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the module-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
2. train, validation, and test split indices at the module level, together with the code required to reconstruct the raw training corpus and cached API responses,
3. trained MENTOR-RL checkpoints for each stage of the pipeline (warm-start SFT, joint closed- and open-book SFT, DPO, and GRPO),
4. the evaluation harness, including MENTOR module holdout benchmarks, out-of-distribution published multiplex benchmarks, and blind comparative evaluation protocols, and
5. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

0.4.4 Broader Impact

By releasing a deterministically-scored, open-source mechanistic exploration agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require proprietary model access. The module-grounded scoring framework is domain-specific, but the underlying pattern, deterministic structured-state rewards over a tool-using policy, is transferable to other scientific settings with curated ground truth.

0.5 Discussion and Conclusion

0.5.1 Reproducibility Without Proprietary Judges

The training pipeline for MENTOR-RL foregoes the use of proprietary judge models to ensure reproducibility and compliance with terms of use. Rewards are computed deterministically over structured state transitions and module-derived ground truth (MENTOR-derived and RWR-LOE) rather than by semantic evaluation from a frontier model. Trajectories are generated either by sampling the policy under training or by deterministic templating, not by proxy labeling through a closed API. One exception is query paraphrasing during dataset construction, for which we will use an open-source model with a pinned version so that the paraphrasing step can be reproduced without API access. Despite avoiding proprietary models during training, we will still compare MENTOR-RL to them through blind comparative evaluation, detailed in Section **??**.

0.5.2 Limitations

Several limitations constrain the scope of the claims we can make with this work.

Reward-hacking risk. The composite module-membership score rewards moves toward the hidden target C. Since C is never observed directly, the agent cannot short-circuit it, but shortcut policies are still possible. For example, a policy that always returns a single-pass random-walk-with-restart expansion from the seed set will recover a substantial fraction of MENTOR modules without genuine multi-step reasoning. The efficiency penalty does not counteract this behavior. We will monitor tool-use entropy and trajectory diversity during training and, if shortcut behavior emerges, introduce adversarially constructed modules or a diversity bonus as mitigations.

Shared-weight verifier. The actor and verifier are two prompting modes of the same policy, so DPO and RL updates on one mode also update the other. This introduces a risk of verifier collapse, in which the verifier learns to endorse the actor's proposals regardless of quality. We will monitor the verifier disagreement rate and, if collapse is observed, consider separate LoRA adapters per mode or a consistency regularizer between modes.

Task difficulty imbalance. The four task types are constructed uniformly during corpus construction, but they are not of equal difficulty; `explanation` tasks require no graph modification, while `refinement` tasks require the verifier to remove members without a tool that directly marks membership. We treat the per-task sampling weights as a hyperparameter and expect to tune them during pilot training.

Split-level leakage. Dendrogram modules share genes across different cuts of the tree; therefore, a module-level split does not by itself prevent leakage through genes shared across overlapping modules. We will report overlap statistics between train and held-out splits and, if overlap is material, adopt a gene-overlap-aware split strategy.

Expert evaluation scope. The blind comparative evaluation relies on 5 to 15 domain experts making pairwise preference judgments. This sample size supports descriptive comparison but not tight confidence intervals on preference rates, and expert rating reliability will be reported alongside preference proportions.

0.5.3 Conclusion

We have presented MENTOR-RL, a proposed open-source reasoning agent for biological mechanistic interpretation, trained through a staged pipeline of supervised fine-tuning, direct preference optimization over shared-prefix trajectory branches, and policy gradient optimization for long-horizon exploration. The training environment grounds rewards in deterministic graph operators and module targets drawn from the MENTOR-derived and RWR-LOE corpora, removing proprietary judges from the loop and exposing every optimization signal as a reproducible, structured quantity. We view MENTOR-RL as one instance of a broader methodological pattern: domain-specialized scientific agents whose behavior is shaped by deterministic structured-state rewards over curated ground truth, rather than by semantic evaluation from frontier models. If successful, this pattern offers a route toward scientific AI systems that are cheap to deploy, auditable in their reasoning, and reproducible across research groups without centralized model access.

Bibliography

[1] Redacted by arXiv. *The Llama 4 Herd: Architecture, Training, Evaluation, and Deployment Notes*. 2026. arXiv: 2601.11659 [cs.SE]. URL: https://arxiv.org/abs/2601.11659.

[2] Akari Asai et al. "Self-rag: Learning to retrieve, generate, and critique through self-reflection". In: *The Twelfth International Conference on Learning Representations*. 2023.

[3] Scott Atchley et al. "Frontier: Exploring Exascale". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: 10.1145/3581784.3607089. URL: https://doi.org/10.1145/3581784.3607089.

[4] Mohammad Gheshlaghi Azar et al. "A general theoretical paradigm to understand learning from human preferences". In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2024, pp. 4447–4455.

[5] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. "Network medicine: a network-based approach to human disease". In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.

[6] Albert-Laszlo Barabasi and Zoltan N Oltvai. "Network biology: understanding the cell's functional organization". In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.

[7] Sumanta Basu et al. "Iterative random forests to discover predictive and stable high-order interactions". In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.

[8] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: https://www.wandb.com/.

[9] Yiqun Chen and James Zou. "Simple and effective embedding model for single-cell biology built from ChatGPT". In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.

[10] Paul F Christiano et al. "Deep reinforcement learning from human preferences". In: *Advances in neural information processing systems* 30 (2017).

[11] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.

[12] Europe PMC Consortium. “Europe PMC: a full-text literature database for the life sciences and platform for innovation”. In: *Nucleic acids research* 43.D1 (2015), pp. D1042–D1048.

[13] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.

[14] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.

[15] Luyu Gao et al. “Pal: Program-aided language models”. In: *International conference on machine learning*. PMLR. 2023, pp. 10764–10799.

[16] Marc Gillespie et al. “The reactome pathway knowledgebase 2022”. In: *Nucleic acids research* 50.D1 (2022), pp. D687–D692.

[17] Madalina Giurgiu et al. “CORUM: the comprehensive resource of mammalian protein complexes—2019”. In: *Nucleic acids research* 47.D1 (2019), pp. D559–D563.

[18] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).

[19] Leland H Hartwell et al. “From molecular to modular cell biology”. In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.

[20] Jiwoo Hong, Noah Lee, and James Thorne. “Orpo: Monolithic preference optimization without reference model”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 11170–11189.

[21] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.

[22] Trey Ideker and Nevan J Krogan. “Differential network biology”. In: *Molecular systems biology* 8 (2012), p. 565.

[23] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: http://www.iso.org.

[24] David Kainer et al. “RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species”. In: *GigaScience* 14 (2025), giaf028.

[25] Sebastian Köhler et al. “Walking the interactome for prioritization of candidate disease genes”. In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.

[26] Takeshi Kojima et al. “Large language models are zero-shot reasoners”. In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.

[27] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: 10.64898/2026.02.27.708412. eprint: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf. URL: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.

[28] Aman Madaan et al. “Self-refine: Iterative refinement with self-feedback”. In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.

[29] Jörg Menche et al. “Uncovering disease-disease relationships through the incomplete interactome”. In: *Science* 347.6224 (2015), p. 1257601.

[30] Marija Milacic et al. “The reactome pathway knowledgebase 2024”. en. In: *Nucleic Acids Res.* 52.D1 (Jan. 2024), pp. D672–D678.

[31] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.

[32] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).

[33] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: https://arxiv.org/abs/2512.20856.

[34] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[35] Rose Oughtred et al. “The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions”. In: *Protein Science* 30.1 (2021), pp. 187–200.

[36] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[37] Shishir G Patil et al. “Gorilla: Large language model connected with massive apis”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 126544–126565.

[38] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).

[39] Yujia Qin et al. “Toolllm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).

[40] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.

[41] Jeff Rasley et al. “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters”. In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.

[42] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.

[43] Timo Schick et al. "Toolformer: Language models can teach themselves to use tools". In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.

[44] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[45] John Schulman et al. "Trust region policy optimization". In: *International conference on machine learning*. PMLR. 2015, pp. 1889–1897.

[46] Zhihong Shao et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models". In: *arXiv preprint arXiv:2402.03300* (2024).

[47] Noah Shinn et al. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.

[48] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: 2601.03267 [cs.CL]. URL: https://arxiv.org/abs/2601.03267.

[49] Avi Singh et al. "Beyond human data: Scaling self-training for problem-solving with language models". In: *arXiv preprint arXiv:2312.06585* (2023).

[50] Kyle A. Sullivan et al. "MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research". In: *bioarXiv* (2024). DOI: 10.1101/2024.07.17.603821.

[51] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: 2507.20534 [cs.LG]. URL: https://arxiv.org/abs/2507.20534.

[52] George Tsitsiridis et al. "CORUM: the comprehensive resource of mammalian protein complexes–2022". In: *Nucleic acids research* 51.D1 (2023), pp. D539–D545.

[53] A. H. Vlot et al. "The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs". In: *PASC 2025*. 2025.

[54] Xuezhi Wang et al. "Self-consistency improves chain of thought reasoning in language models". In: *arXiv preprint arXiv:2203.11171* (2022).

[55] Zhizheng Wang et al. "GeneAgent: self-verification language agent for gene-set analysis using domain databases". In: *Nature Methods* 22.8 (2025), pp. 1677–1685.

[56] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[57] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. https://github.com/huggingface/trl. 2020.

[58] Chunlei Wu, Ian Macleod, and Andrew I Su. "BioGPS and MyGene.info: organizing online, gene-centric information". en. In: *Nucleic Acids Res.* 41.Database issue (Jan. 2013), pp. D561–5.

[59] Shunyu Yao et al. "React: Synergizing reasoning and acting in language models". In: *The eleventh international conference on learning representations*. 2022.

[60] Shunyu Yao et al. "Tree of thoughts: Deliberate problem solving with large language models". In: *Advances in neural information processing systems* 36 (2023), pp. 11809–11822.

[61] Andy B. Yoo, Morris A. Jette, and Mark Grondona. "SLURM: Simple Linux Utility for Resource Management". In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: 10.1007/10968987_3.